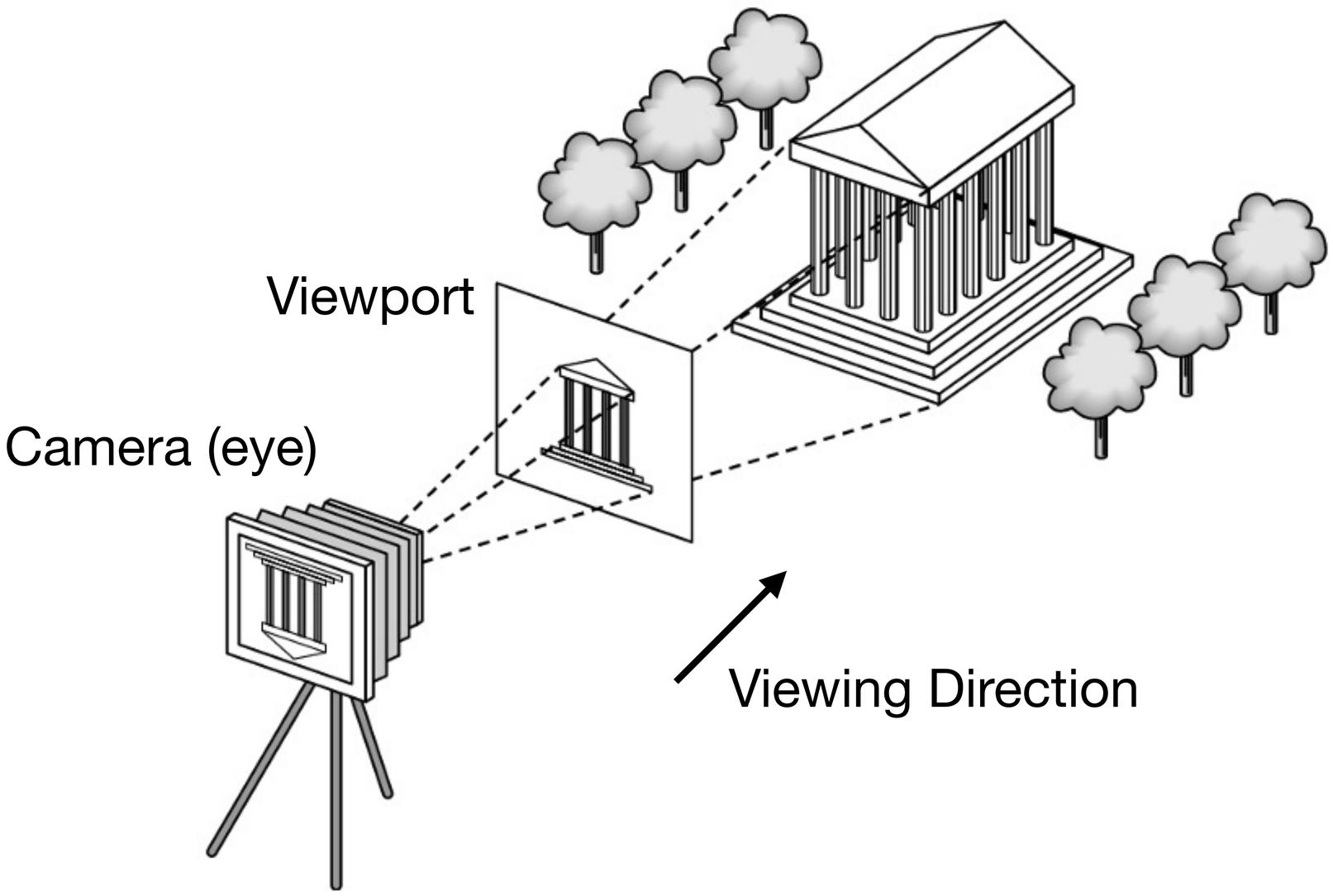
CS 412 / 512
Introduction to Computer Graphics

Zhu Wang
Assistant Professor of Computer Science

COMPUTER SCIENCE

Synthetic Camera



Model matrix

View matrix

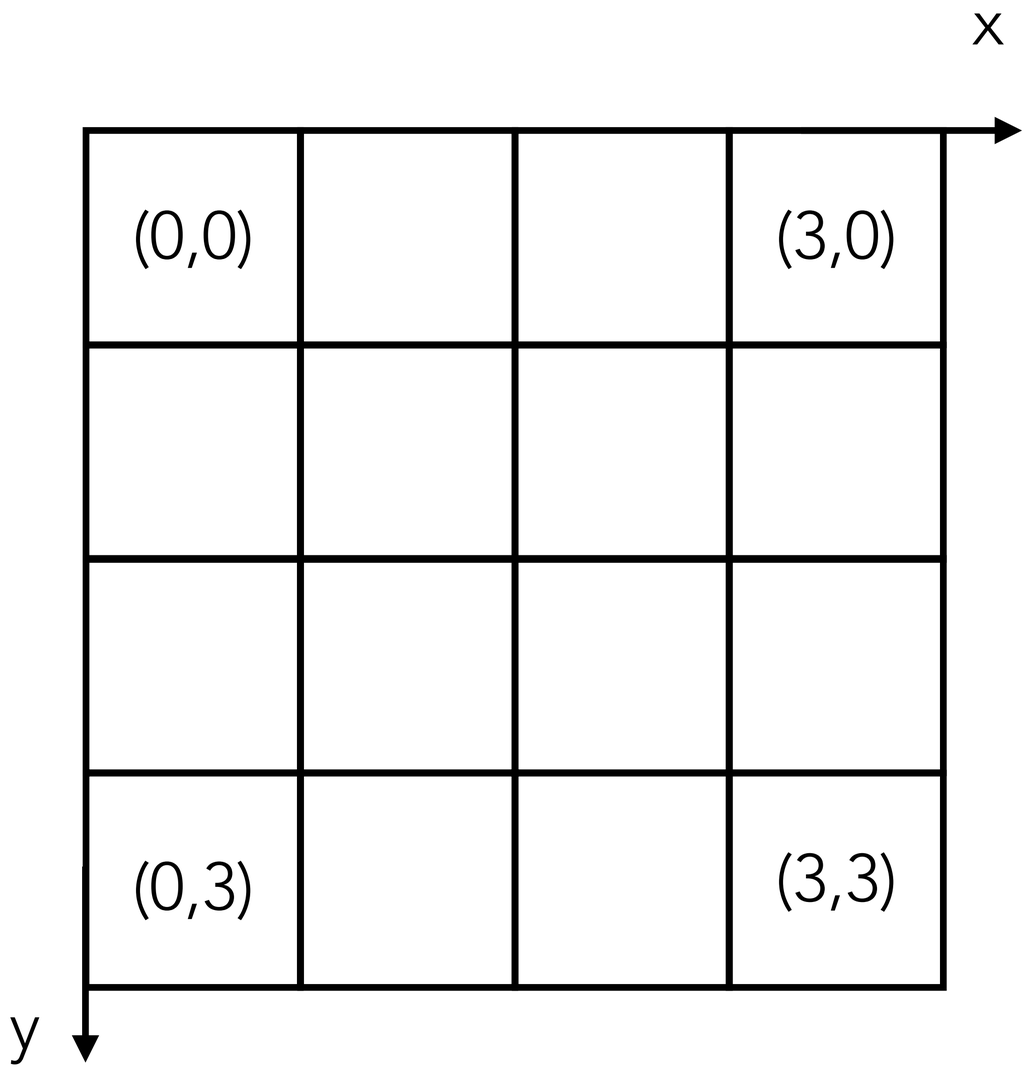
Projection matrix

$$v' = M \cdot V \cdot P \cdot v$$

Image

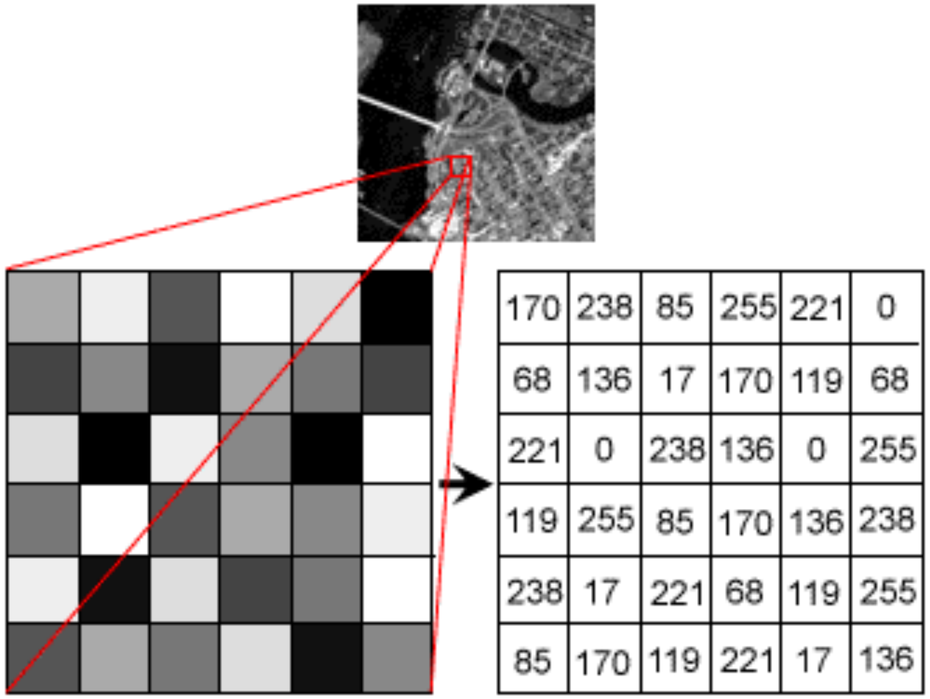
Pixel Coordinates

y axis could be flipped in some coordinate systems



Image

Grayscale image



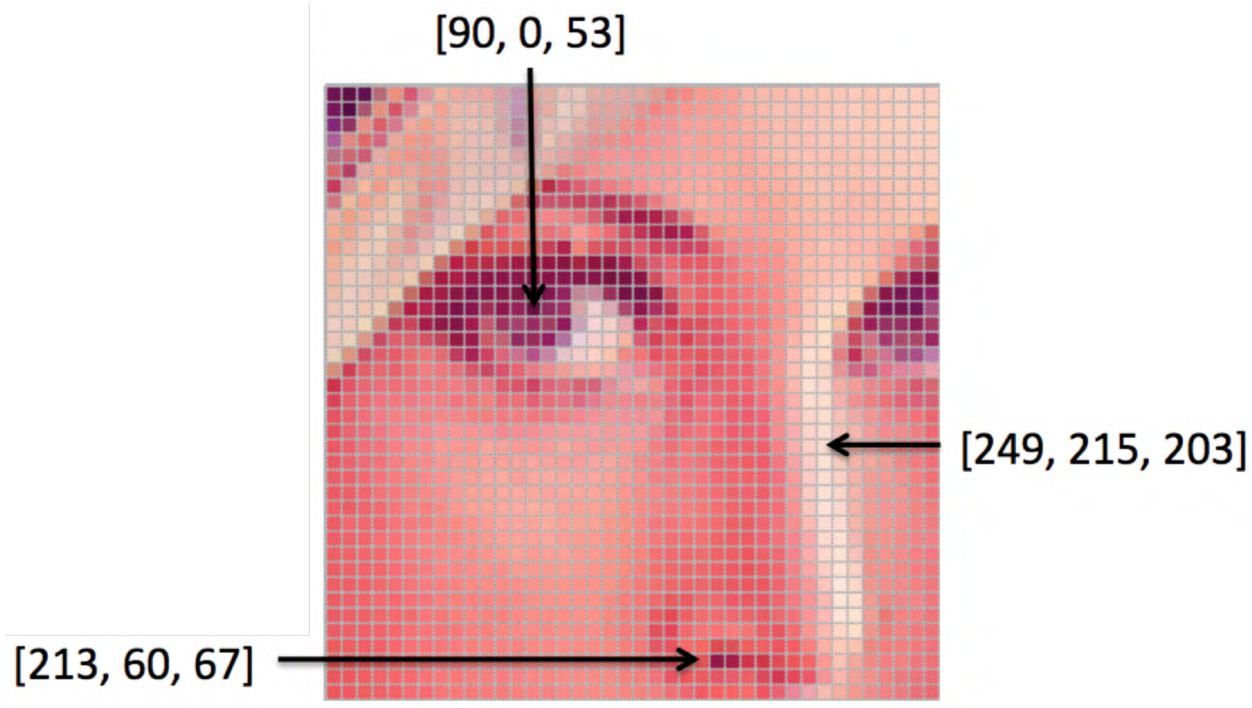
170	238	85	255	221	0
68	136	17	170	119	68
221	0	238	136	0	255
119	255	85	170	136	238
238	17	221	68	119	255
85	170	119	221	17	136

[source: Standford AI Lab]

$$f(x, y) = c$$

0 ~ 255

Color image

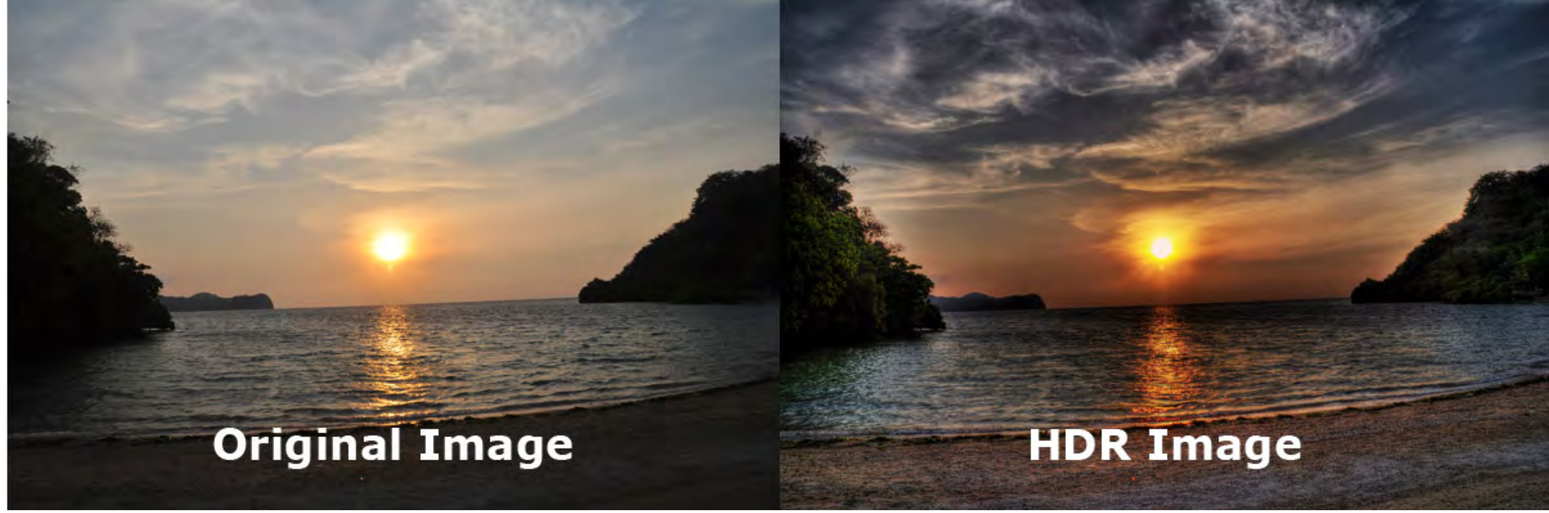


[source: Standford AI Lab]

$$f(x, y) = [r, g, b]$$

0 ~ 255 for each

Image
Pixels and Values



[source: VMI]

For storage and display

Low Dynamic Range (LDR):

- 8 bits (i.e. [0, 255] integer) per channel
- 16 bits (i.e. [0, 65535] integer) per channel

High Dynamic Range (HDR):

- 16-bit half float per channel
- 32-bit float per channel

For computation

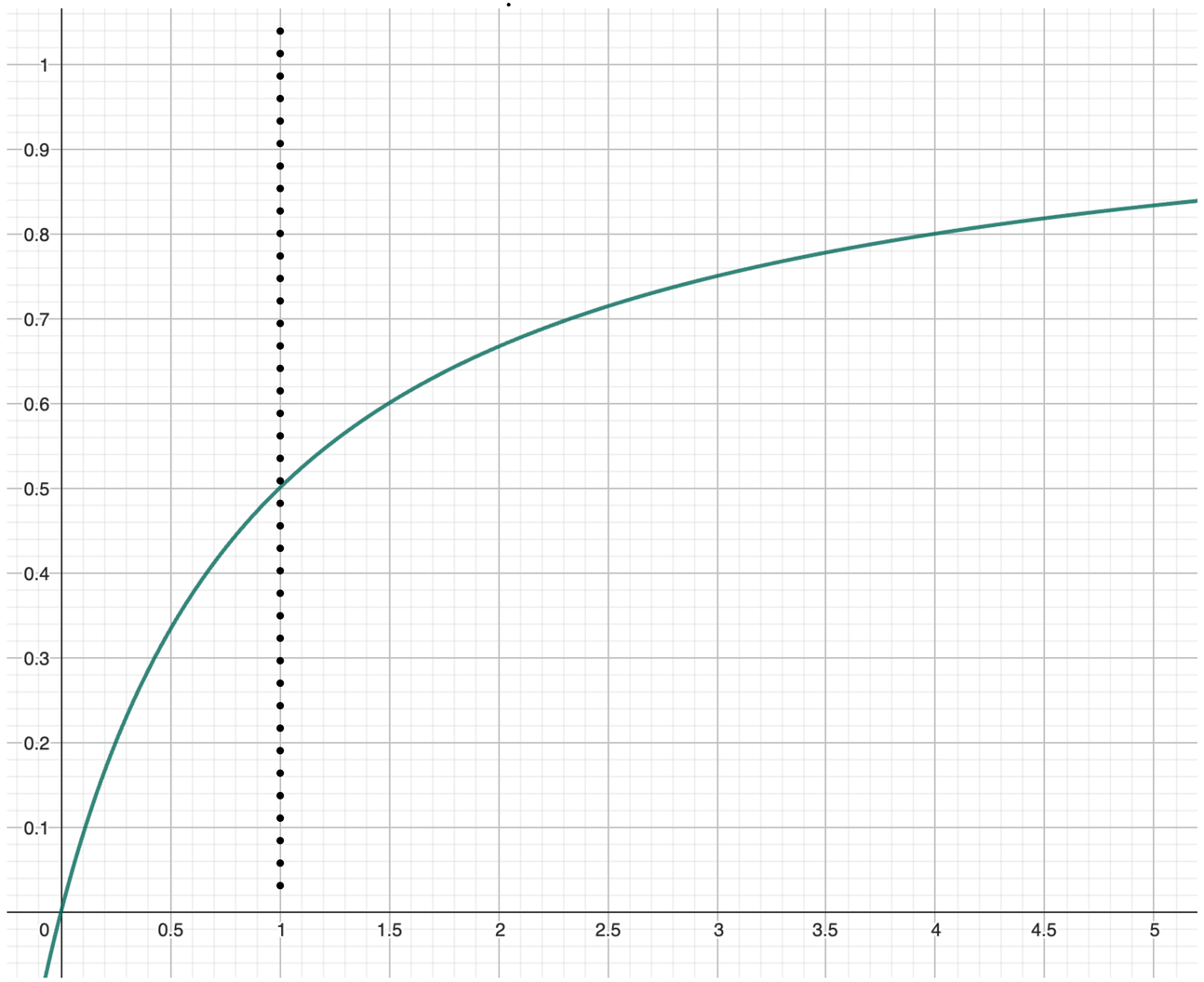
Normalized representation:
range [0, 1] float/double

Tone Mapping

A basic mapping (Reinhard)

$$V' = \frac{V}{1 + V}$$

- $V \ll 1$, mapping is almost linear (dark parts preserved)
- $V \gg 1$, mapping flattens highlights (clipping prevented)



HDR and Multiple Exposures

Exposure Value (EV)

$$V' = V \cdot 2^{EV}$$

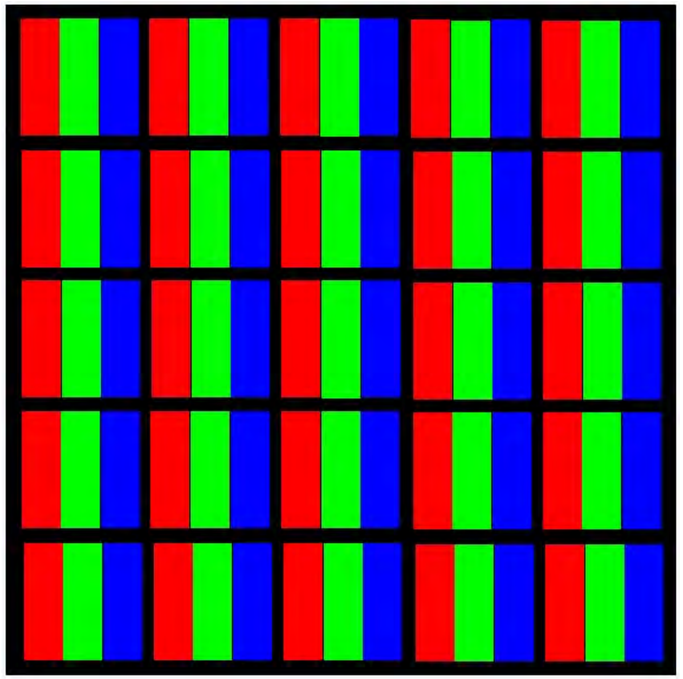
+EV brightens the scene

-EV darkens the scene

[source: Creativeraw]

Image Data Storage

Interleaved



Planar

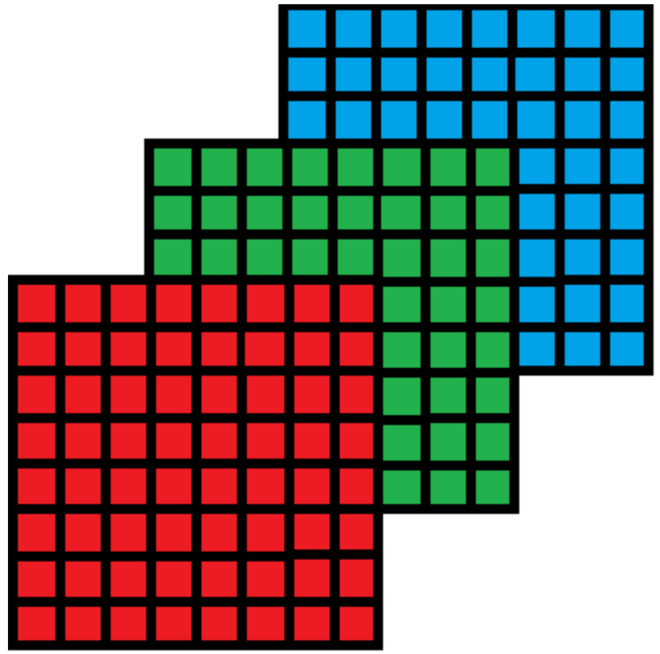


Image Formats

BMP (bitmap image file): mostly uncompressed pixels; no HDR; bigger file sizes.

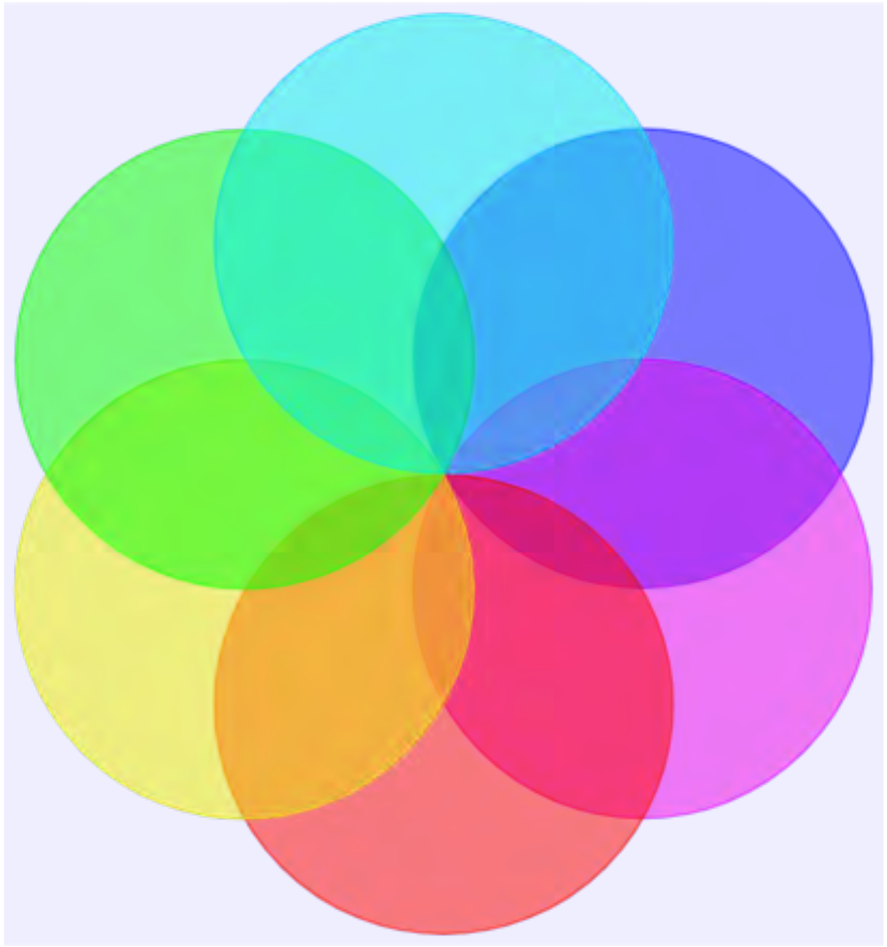
JPEG (Joint Photographic Experts Group): no transparency; lossy compression; limited HDR; compact size.

PNG (Portable Network Graphics): Support transparency and HDR; Lossless compression; larger than JPEG for photos.

TIFF (Tagged Image File Format): Supports lossless compression, multi-layer images, and HDR; Flexible format often used in professional imaging and photography.

EXR (OpenEXR, Extended Dynamic Range): Specialized for HDR; essential for film, VFX, and professional rendering pipelines.

Alpha Channel (α)



Opacity

[0, 1]

α = 0 → 100% transparent

α = 1 → 100% opaque

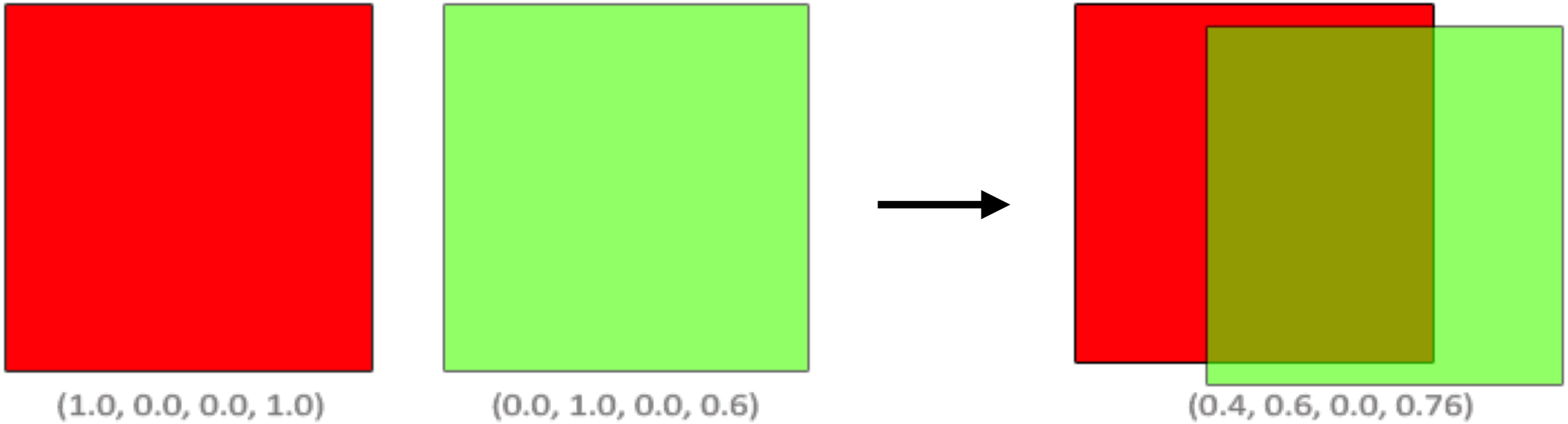
RGB → RGBA

Alpha Blending

Combining a foreground image with a background image using the alpha value

When background is fully opaque

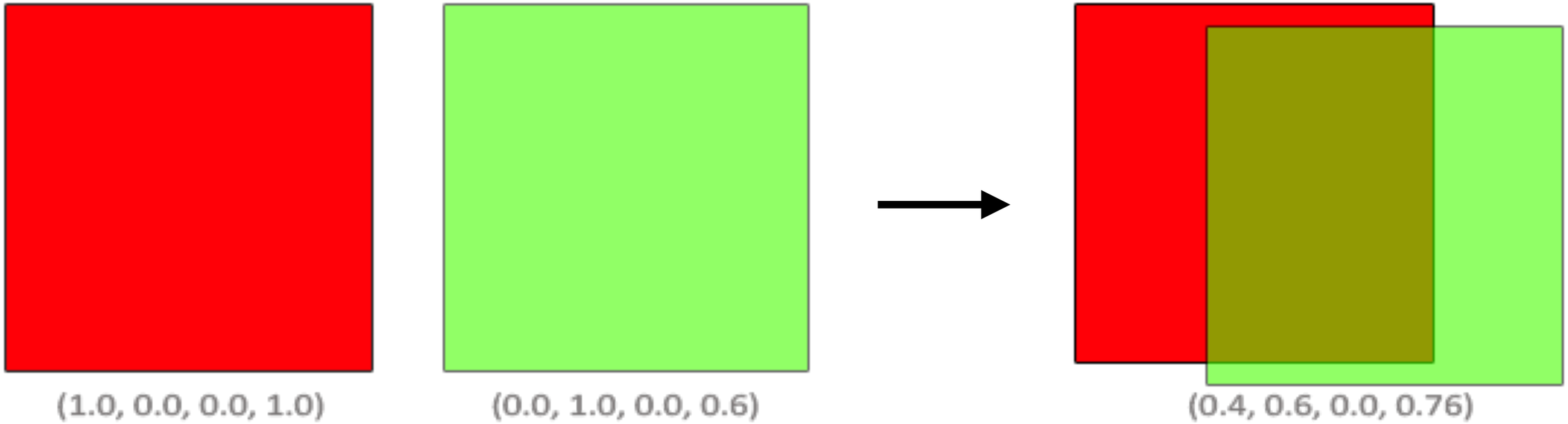
$$C_{final} = \alpha \cdot C_{foreground} + (1 - \alpha) \cdot C_{background}$$



[source: LearnOpenGL]

Alpha Blending

$$C_{final} = \begin{bmatrix} 0.0 \\ 1.0 \\ 0.0 \\ 0.6 \end{bmatrix} \times 0.6 + \begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \\ 1.0 \end{bmatrix} \times (1 - 0.6) = \begin{bmatrix} 0.4 \\ 0.6 \\ 0.0 \\ 0.76 \end{bmatrix}$$



[source: LearnOpenGL]

Alpha Blending

Arbitrary foreground and background alpha

$$\alpha_{final} = \alpha_{foreground} + (1 - \alpha_{foreground}) \cdot \alpha_{background}$$

$$\alpha_{final} \cdot C_{final} = \alpha_{foreground} \cdot C_{foreground} + (1 - \alpha_{foreground}) \cdot \alpha_{background} \cdot C_{background}$$

$$C_{final} = \frac{\alpha_{foreground} \cdot C_{foreground} + (1 - \alpha_{foreground}) \cdot \alpha_{background} \cdot C_{background}}{\alpha_{final}}$$

$$\alpha_{final} = 0 \longrightarrow \alpha_{foreground} = \alpha_{background} = 0$$

Alpha Blending

Premultiplied

$[\alpha R, \alpha G, \alpha B, \alpha]$

- No need to do the multiplications during blending
- Harder to edit RGB

Straight (Unpremultiplied)

$[R, G, B, \alpha]$

- Need to do the multiplications at runtime
- Easier to edit RGB
- Might have interpolation issue

With straight alpha, RGB stores full foreground color independent of alpha. Edge pixels smoothed against a white background carry a baked-in white tint; compositing over black then exposes this as a halo-like outline.

Blending Modes

Additive blending

$$C_{final} = C_{background} + \alpha_{foreground} \cdot C_{foreground}$$

- Foreground brightens the background by adding foreground's color
- In LDR, the resulting intensity is clamped to 1, which can make very bright areas appear as pure white.

[source: Blender]

Blending Modes

Difference blending

$$C_{final} = |\alpha_{foreground} \cdot C_{foreground} - C_{background}|$$

- Matching colors become dark
- Contrasting colors become bright

[source: Sketch]

Blending Modes

Multiply blending

$$C_{final} = C_{foreground} \cdot C_{background}$$

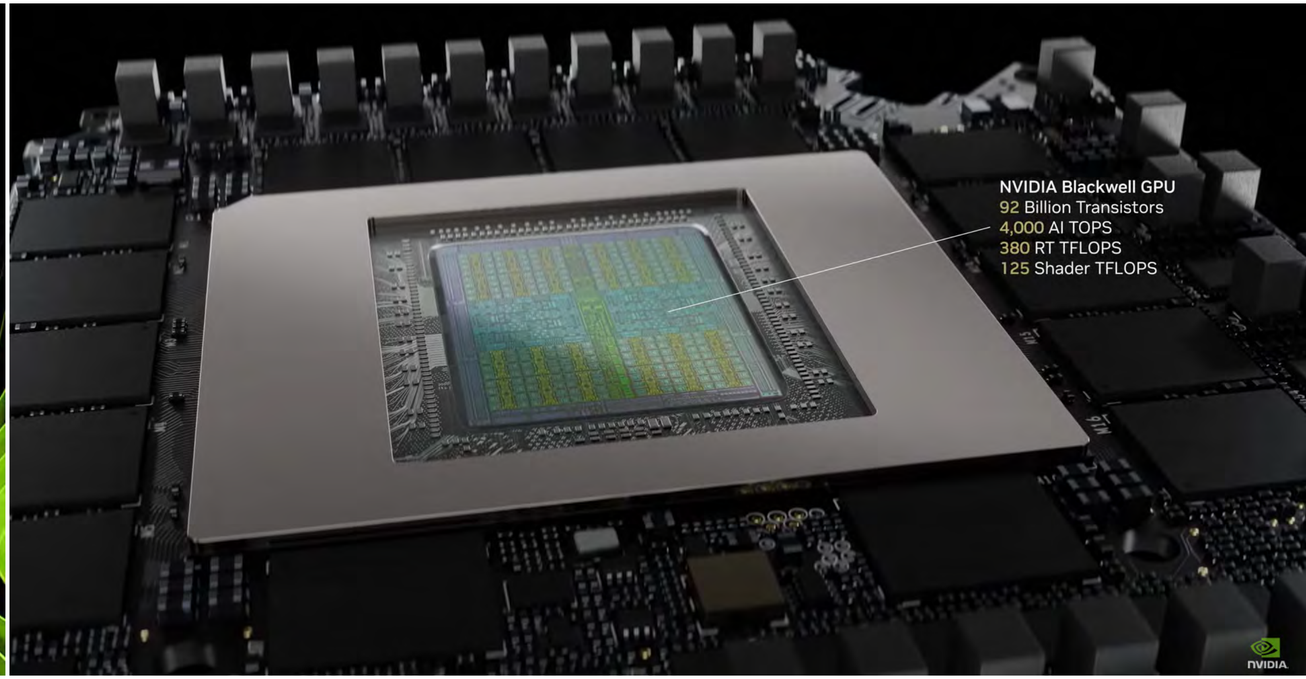
- Black foreground becomes completely black result.
- White foreground makes background unchanged

[source: Elegant Themes]

GPU Pipeline & WebGL

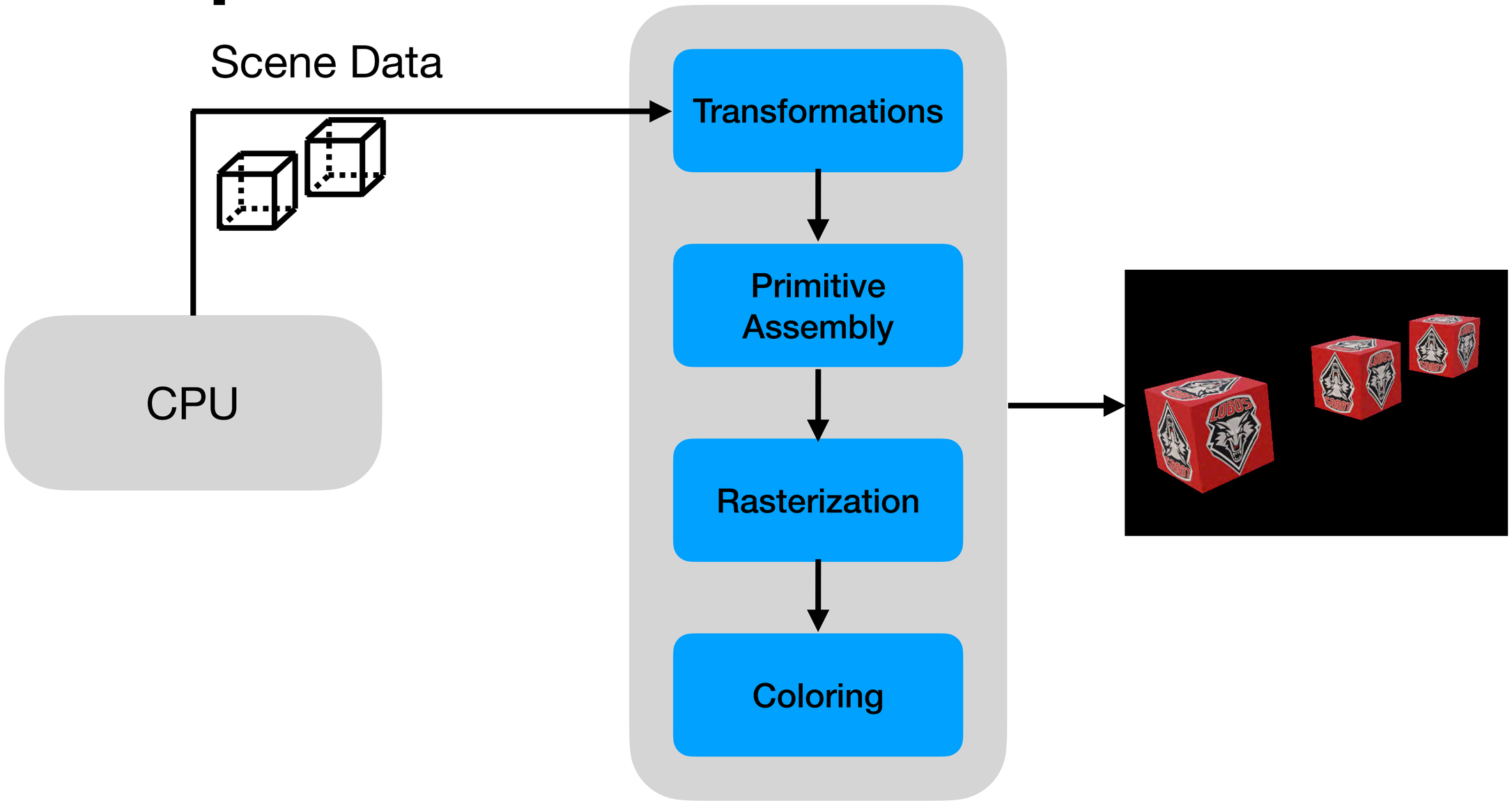
GPU

Graphics Processing Unit

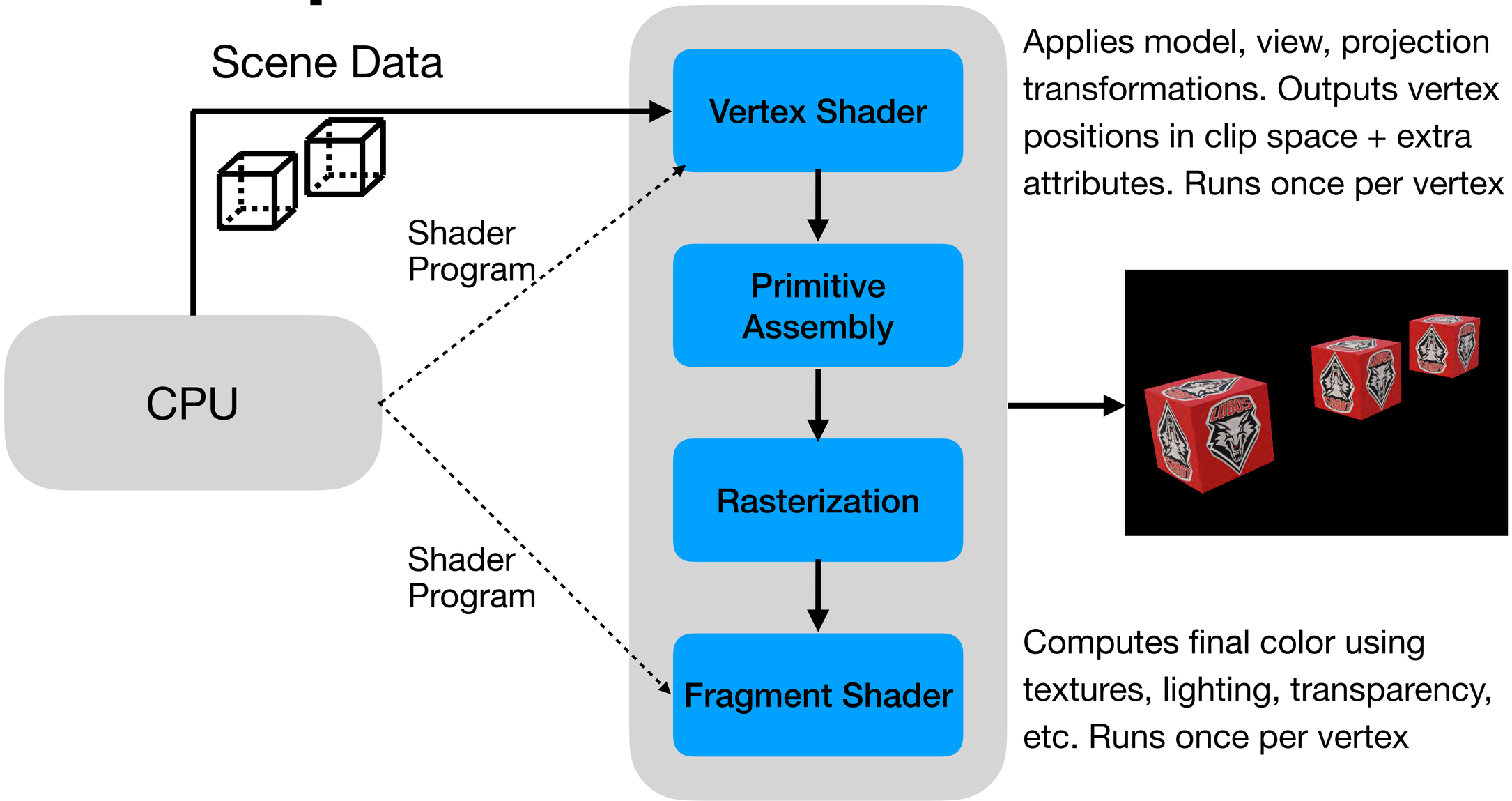


[source: Nvidia]

GPU Pipeline



WebGL Pipeline



WebGL

JavaScript Graphics API

WebGL → OpenGL ES 2.0/3.0 for the web

OpenGL (Open Graphics Library)

Standardized, cross-platform, low-level graphics API

GLSL (OpenGL Shading Language):

The shader language

WebGL2

OpenGL ES 3.0, new syntax, modern features

WebGL

Initialization

```
<canvas id="canvas" width="800" height="600"></canvas>
<script>window.onload = function() {
    const canvas = document.getElementById('canvas');
    const gl = canvas.getContext('webgl2');
    if (!gl)
        { alert("WebGL not supported"); }
    else
        { console.log("WebGL initialized");
```

init()

```
    gl.enable(gl.DEPTH_TEST);
    gl.clearColor(0, 0, 0, 1);
    gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
```

drawScene()

```
} </script>
```

Scene Data

```
Vertices: [
    -1, -1, -1,   // 0
     1, -1, -1,   // 1
     1,  1, -1,   // 2
    -1,  1, -1,   // 3
    -1, -1,  1,   // 4
     1, -1,  1,   // 5
     1,  1,  1,   // 6
    -1,  1,  1    // 7
  ]
```

```
Indices: [
    // front face
    0, 1, 2,  0, 2, 3,
    // back face
    4, 5, 6,  4, 6, 7,
    // left face
    0, 4, 7,  0, 7, 3,
    // right face
    1, 5, 6,  1, 6, 2,
    // top face
    3, 2, 6,  3, 6, 7,
    // bottom face
    0, 1, 5,  0, 5, 4
  ]
```

```
Colors: [
  // Vertex 0: (-1, -1, -1)
  1.0, 0.0, 0.0, 1.0,    // Red
  // Vertex 1: ( 1, -1, -1)
  0.0, 1.0, 0.0, 1.0,    // Green
  // Vertex 2: ( 1,  1, -1)
  0.0, 0.0, 1.0, 1.0,    // Blue
  // Vertex 3: (-1,  1, -1)
  1.0, 1.0, 0.0, 1.0,    // Yellow
  // Vertex 4: (-1, -1,  1)
  1.0, 0.0, 1.0, 1.0,    // Magenta
  // Vertex 5: ( 1, -1,  1)
  0.0, 1.0, 1.0, 1.0,    // Cyan
  // Vertex 6: ( 1,  1,  1)
  1.0, 1.0, 1.0, 1.0,    // White
  // Vertex 7: (-1,  1,  1)
  0.0, 0.0, 0.0, 1.0     // Black
];
```

Buffer Binding

Binding scene data to GPU

```
const positionBuffer = gl.createBuffer();

gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
gl.bufferData(gl.ARRAY_BUFFER,
    new Float32Array(data.vertices), gl.STATIC_DRAW);
```

store vertex attribute data like coordinates, colors, or texture coordinates

```
const indexBuffer = gl.createBuffer();
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, indexBuffer);
gl.bufferData(gl.ELEMENT_ARRAY_BUFFER,
    new Uint16Array(data.indices), gl.STATIC_DRAW);
```

store element indices (vertex indices)

Vertex Shader

```
#version 300 es
in vec3 aPosition;   // vertex position
in vec3 aColor;      // vertex color
uniform mat4 uModelView;    // model transformation
uniform mat4 uProjection;   // projection transformation
out vec3 vColor;     // vertex color

void main() {
   // Apply model and projection transformations
   gl_Position = uProjection * uModelView
                     * vec4(aPosition, 1.0);
   // Pass color data to the fragment shader
   vColor = aColor;
}
```

Fragment Shader

```
#version 300 es
precision mediump float;

in vec3 vColor;         // interpolated from vs
out vec4 fragColor;     // the final pixel color

void main() {
    // Pass color data to the fragment shader
    fragColor = vec4(vColor, 1.0);
}
```

vColor: a varying variable carries per-vertex color data from the vertex shader to the fragment shader.

Resterizaiton:

The GPU takes the triangle defined by 3 vertices. For every pixel (fragment) inside that triangle, it automatically interpolates the vColor values from the vertices.